

2. 데이터분석 AIOps

6. 모델 서빙 및 AIOps 구성

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

1일차

Introducing Model Serving	p.6~40
Introducing Model Serving Patterns	p.41~68
Application 내 추론 런타임 (Scikit-learn)	p.69~76
Application 내 추론 런타임 (PyTorch)	p.78~87
Application 내 추론 런타임 (TensorFlow)	p.89~98
Continuous Model Evaluation	p.100~126
Scikit-learn 추론 런타임 실습	p.77
PyTorch 추론 런타임 실습	p.88
TensorFlow 추론 런타임 실습	p.99

2일차

배치 학습 및 서빙	p.127~136
파이프라인 모델 서빙 - 일반	p.137~174
Ensemble Model Serving Pattern	p.176~187
Business Logic Pattern	p.188~208
전용 Serving 도구 - KServe	p.209~230
전용 Serving 도구 - vLLM	p.232~250
파이프라인 모델 서빙 실습	p.175
KServe 실습	p.231
vLLM 실습	p.251

3일차

MLflow	p.252~277
Observability - 일반	p.281~298
LGTM	p.299~333
DevOps	p.335~346

MLOps	p.347~355
LLMOps	p.356~388

MLflow 실습	p.278
LGTM 실습	p.334
LLMOps 실습	p.389
평가 (퀴즈)	

제외 항목

전용 Serving 도구 - Ray Serve	p.391~410
Kubeflow	p.411~434
LangSmith	p.435~448



1~4교시(중계)

Introducing Model Serving
Introducing Model Serving Pattern
Scikit-learn
Continuous Model Evaluation

5~6교시(실습교수, 실습 및 보충강의)

PyTorch·Keras는 중계에 없으므로 실습교수가 이론까지 보충
실습 6개 진행.

7~8교시(실습교수, 자율학습 및 Q&A)

각 예제 폴더의 followup.md를 그대로 따라가며 자율 진행
강사는 순회하며 질의응답

선수지식

직렬화(Serialization)
신경망 기초 개념
동시성/멀티프로세스에 대한 기본 개념
가상환경/패키지 관리 개념

학습된 모델 객체(트리 구조, 가중치)를 파일로 저장했다가 재학습 없이 복원
MLP(다층 퍼셉트론), 역전파알고리즘
GIL(Global Interpreter Lock)
전부 uv venv 방식
uv는 Astral이라는 회사가 Rust로 만든 도구로,
venv+ pip+ pip-tools+ pyenv 역할을 하나로 합친 올인원 패키지/프로젝트 관리자다.

기본 실행 환경

Python 3.11
uv
curl
lsuf

uv venv --python 3.11 --clear .venv로 새 가상환경을 만든다.
FastAPI fastapi-sklearn/fastapi-pytorch/fastapi-keras /health, /predict 호출
lsuf -ti:포트 | xargs kill -9
백그라운드로 띄운 uvicorn 서버를 포트 기준으로 찾아 종료

핵심 ML/서빙 라이브러리

scikit-learn 1.9.0
numpy
torch 2.14.0

keras 3.15.1

tensorflow 2.21.0
fastapi 0.141.1

uvicorn[standard] 0.52.4
pydantic

Iris 데이터 로드(load_iris)
배열 연산 및 .npz 파일 저장/로드(테스트셋, 스케일러 파라미터)에 사용.
base-pytorch/fastapi-pytorch
IrisNet(nn.Module) 정의, 학습, torch.jit.script를 이용한 TorchScript 컴파일에 사용.
base-keras/fastapi-keras
os.environ["KERAS_BACKEND"] = "tensorflow"로 백엔드를 먼저 고정
keras 3의 실행 백엔드로 사용
fastapi-sklearn/fastapi-pytorch/fastapi-keras 서빙 프레임워크.
lifespan으로 모델을 프로세스 시작 시 한 번만 로드하는 패턴
FastAPI 앱을 실제로 구동하는 ASGI 서버.
FastAPI가 내부적으로 요청 바디 검증(BaseModel, Field)에 사용하는 라이브러리

표준 라이브러리

별도 설치 불필요, 코드에서 직접 활용

pickle base-sklearn/fastapi-sklearn에서 모델 저장/복원에 사용.
multiprocessing, urllib.request, statistics, time, json fastapi-pytorch의 perf_test.py가 외부 부하테스트 도구(locust 등) 없이
직접 만든 동시성 부하 테스트 클라이언트다.

Model Serving

Data preparation (데이터 준비)

원천 데이터를 모으는 단계.

Preprocessing(전처리)
Feature Engineering(피처 엔지니어링)

결측치 처리, 정규화 등 모델이 바로 쓸 수 있는 형태로 데이터를 다듬는 단계.
원본 데이터에서 모델 성능에 도움이 되는 파생 변수(피처)를 만들어내는 작업.

Airflow

정해진 순서(DAG)대로 자동 실행/스케줄링하는 워크플로우 오케스트레이터.
데이터 준비 단계뿐만 아니라 Model Build 단계의 파이프라인도 같은 도구로 자동화

Model Build (모델 빌드)

Data Feature

앞 단계에서 만든 피처를 모델 학습 입력으로 가져오는 부분.

Train / Fine tuning
Evaluate
Pipeline / MLOps Framework
Airflow
MLflow
Kubeflow
Ray Serve

모델을 처음부터 학습시키거나 학습된 모델을 특정 목적에 맞게 추가 학습
학습된 모델의 성능을 지표(accuracy 등)로 측정하는 단계.
Data Feature→Train→Evaluate를 자동화된 파이프라인으로 엮어주는 프레임워크
범용 워크플로우 오케스트레이터 도구. ML 전용은 아님
실험 추적+ 모델 등록에 특화된 도구.
Kubernetes 위에서 ML 파이프라인 전체를 관리하는 플랫폼.
정확히는 분산 서빙 프레임워크 Ray의 Serve 모듈.
Model Serving(서빙) 역할에 더 가까운 도구

Model Release (모델 릴리즈)

Artifact Store
Model Registry

학습된 모델 파일, 로그, 그래프 같은 산출물을 저장하는 저장소.
등록된 모델의 버전을 관리하는 저장소.
MLflow라는 도구 하나가 이 두 기능을 한꺼번에 제공

Model Serving (모델 서빙)

Serving Framework

학습이 끝난 모델을 실제 요청에 응답하는 서비스로 만드는 프레임워크 계층.

FastAPI
KServe(Kubeflow)
vLLM
BentoML

파이썬 웹 프레임워크. 직접 서빙 로직을 작성하는 방식.
Kubernetes 전용 모델 서빙 플랫폼.
LLM 전용 고성능 서빙 엔진.
모델을 패키징하고 API 서버까지 자동 생성해주는 프레임워크로,

FastAPI가 "직접 다 짜야 하는" 방식이라면
BentoML은 "모델 파일 하나 던지면 서빙 API를 자동으로 만들어주는" 규격화된 방식

Model Runtime	모델을 실제로 실행(추론)하는 엔진 계층.
Onnx Runtime	ONNX 표준 포맷으로 변환된 모델을, PyTorch나 TensorFlow 같은 원래 프레임워크 없이도 실행할 수 있게 해주는 별도의 경량 추론 엔진
Model Format	모델을 저장하는 파일 형식.
Onnx	Onnx Runtime이 읽는 표준 모델 파일 포맷
	프레임워크 간 호환을 위한 공용 포맷
Bento	BentoML 전용 패키징 포맷.
	모델 파일뿐 아니라 실행 환경(의존성)까지 하나로 묶는다.

Monitoring (모니터링)

Observability(Metrics/Tracing/Logs)	도구 LGTM(Loki/Grafana/Tempo/Mimir)
Quality Drift	모델이 배포된 이후 시간이 지나며 입력 데이터 분포나 모델 성능이 처음과 달라지는 현상(데이터/모델 드리프트)을 감지하는 영역.
	도구 LangSmith

서비스 영역

(Service) Frontend / (Service) Backend	모델 서빙 결과를 실제로 사용자에게 보여주는 애플리케이션 계층 웹/앱 화면과 그 뒤의 백엔드 서버
AI Agent(s)	여러 도구·모델 호출을 스스로 판단해 순차적으로 실행하는 자율 에이전트.
RAG Agent	검색 증강 생성을 수행하는 에이전트. 질문이 들어오면 벡터 DB 등에서 관련 문서를 먼저 검색(Retrieval)한 뒤, 그 내용을 근거로 LLM이 답을 생성(Generation)하는 구조다.

Deployment Environments

검증 없이 운영에 바로 배포하던 시절, 변경 하나가 실사용자 장애로 직결되던 문제
환경 계층이 없으면 여러 개발자의 변경을 합쳐서 검증할 무대도, 운영과 같은 조건으로 리허설할 무대도 없다.
단계마다 다른 결함을 조기에 걸러내기 위해

로컬 Local	기능 구현·단위테스트
개발 Dev	변경 통합
스테이지 Staging	운영과 같은 조건 QA
운영계 Prod	승인 후 배포

Serving 개념

BentoML 예시

학습을 끝낸 모델은 그 자체로 아무도 접근할 수 없는 파일에 불과
"서빙한다"는 말이 저장/접근점/배포/버전관리 중 무엇을 가리키는지 정리가 안 되면 소통에 혼선

저장→접근점→배포→버전관리 4단계로 표준화

Model Store

태그·버전으로 모델을 추적하는 저장소(예: bentoml models list)

Deployment Artifact

코드+모델+의존성을 하나로 패키징해, 어느 환경에 배포하든 동일 결과 보장
(예: bentofile.yaml 선언 → bentoml build)

Model-serving Tools 지형도

매번 처음부터 만들 필요는 없다.
모르면 시간·리스크가 늘어난다.
요구사항에 맞는 도구 선택

Flask/FastAPI(DIY)

범용 웹 프레임워크를 가져다 직접 서빙 로직을 짜는 방식.
학습 곡선이 없고, 완전히 원하는 대로 커스터마이징할 수 있다.
버전 관리, 배치, 오토스케일링, 모니터링을 전부 직접 구현해야 하고,
상태를 유지(stateful)해야 하는 서빙에는 부적합하다.

TensorFlow Serving

프레임워크 전용 단일 모델 서버다.
Docker 이미지 하나로 서빙할 수 있고, model.config 파일만으로 다중 모델·다중 버전을 동시에 서빙할 수 있다.
기본적으로 SavedModel 포맷 전제이며, 다른 프레임워크의 모델은 변환하거나 래핑해야 한다.

TorchServe

프레임워크 전용 단일 모델 서버다.
handler(전처리/후처리 로직)만 작성하면 .mar 패키징만으로 REST/gRPC 엔드포인트가 즉시 노출되고,
다중 모델·버전 관리·자동 배치가 기본 제공되며, AWS와 Meta가 공식 지원한다.
handler를 직접 작성해야 해서 초기 학습 곡선이 있고, TensorFlow Serving만큼 축적된 대규모 운영 사례는 적다.

BentoML

여러 프레임워크를 통합하는 SDK이자 배포 아티팩트 포맷이다.
save_model() 하나로 15개 이상의 프레임워크를 통일된 API로 다룰 수 있고,
코드+모델+의존성을 Bento라는 산출물 하나로 패키징해서 컨테이너화나 클라우드 배포까지 일원화할 수 있다.
자체 Runner/Service 추상화 개념을 새로 익혀야 하고,
지원 범위 밖의 커스텀 모델은 어댑터를 따로 만들어야 한다.

KServe

Kubernetes 네이티브 서빙 플랫폼이다.
InferenceService라는 선언 하나로 오토스케일링이 되고,
카나리 트래픽 분할이 가능하며, 프레임워크들을 통일된 인터페이스로 다룰 수 있다.
Kubernetes와 Knative가 반드시 있어야 하고, 초기 러닝커브가 크며, 단순한 배치 추론에는 과한 도구다.

Ray Serve

분산 실행 엔진 기반 서빙이다.
@serve.deployment로 컴포넌트별로 독립적으로 스케일링(GPU/CPU 자원을 분리)할 수 있고,
.bind()로 여러 컴포넌트를 DAG 형태의 파이프라인으로 구성할 수 있으며, 요청 배치를 지원한다.
Ray 클러스터 자체를 운영하는 오버헤드가 있고, GPU replica는 예약하는 즉시 과금되어 유틸 비용이 생기며,
콜드스타트 지연이 있다.

MLflow

서빙이 일부 기능일 뿐인 MLOps 플랫폼이다.
실험 추적과 모델 레지스트리, 배포를 한 플랫폼에서 관리할 수 있어 재현성과 버전 추적이 강력하다는 것이다.
단점은 Tracking Server를 고가용성으로 구성하려면 별도 백업 구성이 필요하고,
대규모 아티팩트에는 S3 같은 오브젝트 스토리지가 필수이며,
정교한 데이터 리니지 추적은 지원 범위 밖이라는 것이다.

Kubeflow

Kubernetes 네이티브 ML 파이프라인 플랫폼이며, 서빙 자체는 KServe에 위임하는 구조다.
학습(Training Operator), 튜닝(Katib), 서빙(KServe)까지 전체 ML 워크플로우를 관리할 수 있다.
서빙 기능을 자체적으로 내장하지 않고 KServe에 위임한다는 것 자체가 구조적 제약이 될 수 있다.

ONNX(공통 저장 포맷)

학습 프레임워크(PyTorch)와 서빙 프레임워크가 실제로 서빙되는 도구(예: 특정 벤더 런타임)와 서로 다를 수 있다.
프레임워크 고유 포맷만 쓰면 배포 시 그 학습 프레임워크에 고착(종속)된다.
ONNX 같은 교환 포맷이 없으면 PyTorch로 학습한 모델을 TensorFlow Serving이나

다른 런타임에 넘길 표준 방법이 없어 매번 변환 코드를 새로 짜야 한다.

.onnx 파일 하나로 여러 서빙 도구에 동일 모델을 전달한다.

실제로 `torch.onnx.export(model, dummy_input, "alexnet.onnx")`로 PyTorch AlexNet을 .onnx로 내보내고, 같은 .onnx 파일을 PyTorch 프로세스와 ONNX Runtime 프로세스 양쪽에서 각각 로드해 추론했더니 정확도가 0.9667로 완전히 일치한다.

Bento

서빙하려면 모델 파일뿐 아니라 실행 코드·의존성·설정까지 함께 필요한데, 이게 매번 다른 방식으로 묶이면 재현이 안 되고 환경마다 배포·조회 명령이 달라진다.

bentofile.yaml 하나에 서비스 코드·모델·의존성·설정을 선언하면, `bentoml build`로 모델·코드·의존성을 묶은 Bento(NAME:TAG로 식별되는 배포 단위)가 만들어지고, `bentoml list`로 조회, `bentoml serve NAME:TAG --production`으로 즉시 프로덕션 API를 띄울 수 있다.

책임 단위 분리

Model Serving은 데이터 과학자와 엔지니어가 서로 다른 도메인 지식을 갖고 협업해야 하는데, 역할 경계가 불분명하면 저장 형식·모델 호환성·버전 관리·엔드포인트 노출 같은 질문에 누가 책임지는지가 불분명해진다.

저장 형식·모델 호환성·버전 관리·엔드포인트 노출 같은 질문이 누구 담당인지 불분명해지면 시빙 자체가 지연된다. "역할·팀 단위"로 명확한 담당 영역을 나눈다.

Model Versioning

모델은 CI/CD 파이프라인만으로는 부족하다.

새 버전을 손쉽게 재배포할 수 있으면 ML 모델은 그럴 만큼 단순하지 않다.

모델 버전 관리 없이 "어떤 모델이 어떤 데이터·피처로 학습되었고 어떤 성능이었는지"를 남기지 않으면, 성능이 저하됐을 때 원인을 특정할 방법이 없다.

`register_model_version()` 같은 함수로 버전 등록 시 피처·재학습 데이터·평가 지표를 함께 기록해두면, 재학습·재배포를 거친 모델마다 버전을 부여해 이전 결과와 비교하고, 성능이 나빠진 버전을 롤백하거나 하위 호환성이 깨지는 시점을 식별할 수 있다.

Model Serving Patterns

서빙 철학 3대 원칙

Stateless(무상태)

Backend는 항상 켜져 있다"는 암묵적 가정

스턴스마다 상태를 직접 들고 있으면 재시작·확장 시 일관성이 깨짐

외부 저장소에 상태를 위임

Continuous Model Evaluation

배포 후에도 계속 성능을 모니터링하는 원칙(4번 항목에서 심화)

Keyed Prediction

병렬 서버 응답이 요청 순서와 뒤섞임

순서만으로는 어떤 응답인지 알 수 없음

key로 요청·응답을 정확히 매핑

6대 서빙 패턴

Batch

야간 배치 스코어링처럼 응답을 즉시 줄 필요가 없을 때 쓴다.

HTTP 요청-응답 구조로는 대량 예측이 타임아웃에 걸려 불가능하므로,

예측 대상 전체를 모아 비동기로 한 번에 처리하고 결과를 파일/DB에 적재하는 방식이다.
지연시간 대신 처리량을 최적화한다.

Online

챗봇처럼 요청마다 즉시 동기 응답이 필요할 때 쓴다.

클라이언트가 REST/gRPC로 요청을 보내면 서버가 같은 커넥션 안에서 바로 결과를 반환하며,
점진적 출력이 필요하면 WebSocket 스트리밍으로 확장한다.

미리 계산해둘 수 없는 다양한 질의를 다뤄야 할 때의 기본 형태다.

Two-phase

엡지 디바이스가 네트워크 불안정으로 서버 모델에 항상 접근할 수 없을 때 쓴다.

엡지에 가볍고 정확도가 낮은 phase one 모델을 두어 오프라인에서도 기본 예측을 내주고,
네트워크가 연결되면 서버의 무거운 phase two 모델이 더 정밀한 예측으로 이어받는다.

후보 생성 후 정밀 재순위화가 필요한 케이스에도 같은 구조가 쓰인다.

Pipeline

이미지 전처리→객체 검출→캡션 생성처럼 하나의 모델(함수)이 감당하기 어려운 다단계 작업일 때 쓴다.
전체를 하나로 뭉쳐두면 단계 하나만 교체·개선하려 해도 전체를 다시 만들어야 하고 병목도 못 찾는데,
각 단계를 독립된 ML 태스크로 분리하면 단계별 교체와 최적화가 가능해진다.

Ensemble

신모델을 배포할 때 구모델을 바로 끄기 불안하거나, 입력 특성별로 다른 특화 모델이 필요할 때 쓴다.

신·구 모델을 동시에 서빙해 안전망을 유지하면서,

여러 모델 출력을 평균·투표로 집계하거나 입력에 따라 동적으로 모델을 골라 호출한다.

Business logic

추론 서버가 예측 외에 자격 확인·입력 검증·전처리까지 떠안는 문제를 해결하기 위한 패턴이다.

이게 없으면 미인가 사용자 요청이나 DDoS성 요청이 그대로 추론 서버까지 도달해 자원을 낭비하므로,
검증·전처리를 별도 계층으로 분리해 추론 서버는 순수하게 예측만 담당하게 한다.

Scikit-learn

base-sklearn

[base-sklearnWfollowup.md](#) 순서대로 진행

서버 없이 pickle 저장·복원 비교

RandomForestClassifier를 학습해 pickle로 저장한 뒤, 완전히 새로운 python 프로세스에서
그 파일만으로 모델을 복원해 predict까지 실행해 본다.

"다시 학습하지 않고도, 저장된 pickle 파일만으로 학습 직후와 똑같은 결과를 재현할 수 있는가?"

판정 기준은 두 프로세스의 테스트 정확도가 소수점까지 정확히 일치하는지 하나뿐이다.

지도학습 분류(Classification)의 기본 개념

결정트리·앙상블

객체 직렬화(Serialization)란 무엇인가

프로세스(Process)와 메모리 개념

학습(fit)과 추론(predict)이 별개 단계라는 것

RandomForest가 여러 트리의 다수결로 예측한다는 정도의 개념

메모리 상의 객체를 파일로 저장했다가 나중에 그대로 복원하는 개념

프로세스가 종료되면 그 메모리에 있던 변수도 함께 사라진다는

운영체제 수준의 기본 이해

"왜 별도 프로세스에서 재학습 없이 복원되는 게 신기한 일인지"

파이썬 객체를 바이트 스트림으로 직렬화(serialize)해서 파일이나 네트워크로
저장·전송할 수 있게 한다.

원래 객체로 복원하는 걸 역직렬화(unpickling)라고 한다.

표준 라이브러리 pickle 모듈이 이 기능을 제공한다.

피클링(pickling)

가상 환경 생성

`cd base-sklearn`

실습 전용 폴더로 이동

```
winget install --id=astral-sh.uv -e
이 폴더 전용 가상환경을 새로 만들.
```

```
--clear
```

혹시 전에 만들어둔 .venv가 있으면 지우고 새로 시작

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass      보안오류
.\venv\Scripts\WActivate.ps1                                     .venv 실행
```

방금 만든 가상환경을 현재 셸에 활성화.

이후 python·pip 명령이 시스템 전역이 아니라 이 .venv 안의 것을 가리게 됨

이 실습에 필요한 라이브러리만 설치(scikit-learn, numpy).
--quiet 설치 로그를 최소한만 출력.

```
print('scikit-learn', sklearn.__version__); print('numpy', numpy.__version__)"
설치가 제대로 됐는지 버전을 즉석에서 확인.
-c "... "는 별도 .py 파일 없이 그 자리에서 문자열로 코드를 바로 실행하는 옵션
```

"별칭 무시하고 PATH에 있는 진짜 실행 파일 써라"는 셸 내장 명령
macOS alias python=... 문제 회피용이라, 이 alias 우회 없이 그냥 python만 쓰면
사람에 따라(특히 macOS에서 zsh alias를 걸어둔 학생) .venv가 아니라
영동한 인터프리터가 실행될 수 있다.

scikit-learn으로 붓꽃(Iris) 분류 모델을 학습하고 pickle로 저장한다.
"다시 학습하지 않고도, 저장된 pickle 파일만으로 학습 직후와 똑같은 결과를 재현할 수 있는가?"

```
import pickle                # 학습된 모델 객체를 파일로 직렬화(저장)/역직렬화(로드)하기 위한 표준 라이브러리
import time                  # 전체 실행 시간을 재기 위해서만 사용 (학습 로직과는 무관)
import numpy as np           # 테스트셋을 .npz로 저장할 때 사용
from sklearn.datasets import load_iris          # Iris 데이터셋 로드. scikit-learn에 내장
from sklearn.ensemble import RandomForestClassifier # 사용할 분류 모델
from sklearn.model_selection import train_test_split # 학습/테스트 데이터 분리 유틸
```

```
def main() :
    # 1) 데이터 준비 — Iris 데이터셋(150 샘플, 4 특성, 3 클래스)
    X, y = load_iris(return_X_y=True)
    # X: 4개 특성(꽃받침·꽃잎 길이·너비), y: 품종 레이블(0/1/2)
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=0, stratify=y
    )
    # test_size=0.2          150개 중 20%(30개)를 테스트용으로 떼어놓음
    # random_state=0         분리 결과가 실행할 때마다 똑같이 재현되도록 시드 고정
    # stratify=y             품종 비율이 train/test에 원본과 같은 비율로 유지되도록 층화 추출
)
```

```
# 2) 학습 — SCIKIT-LEARN.md 예시와 동일한 두 줄 패턴
model = RandomForestClassifier(random_state=0) # 모델 객체 생성
model.fit(X_train, y_train) # 실제 학습.
```

```
# 3) 학습 직후(같은 프로세스) 정확도 확인 — 2단계 결과와 비교할 기준값
pred_inproc = model.predict(X_test) # 방금 학습한 model로 테스트셋 30개 예측
acc_inproc = (pred_inproc == y_test).mean() # 예측값과 실제 레이블이 일치하는 비율 = 정확도
```

```
# 이 값이 나중에 predict_from_pickle.py(별도 프로세스)에서 나온 정확도와 정확히 같은지 비교할 "기준값"이 된다.
print(f"[학습 직후] 테스트 정확도: {acc_inproc:.4f} ({(pred_inproc == y_test).sum()}/{len(y_test)})")
```

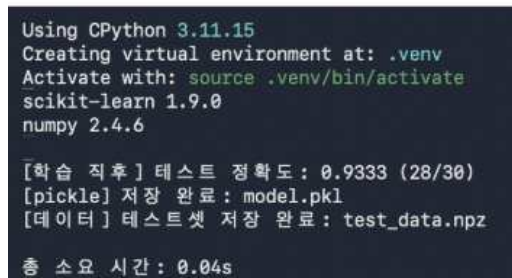
```
# 4) pickle로 저장 — 학습된 model 객체(트리 구조·가중치 전부)를 그대로 직렬화
model_path = "model.pkl"
with open(model_path, "wb") as f:                                # pickle은 텍스트가 아니라 바이너리 포맷
    pickle.dump(model, f)                                         # model 객체 전체를 f에 직렬화해서 씬
print(f"[pickle] 저장 완료: {model_path}")
```

```
# 5) 다음 단계(predict_from_pickle.py)에서 그대로 재사용할 수 있도록 테스트 데이터 저장
# test_data.npz 하나에 X_test·y_test·acc_inproc을 각각 이름 붙여서 저장
# 다음 스크립트가 이 파일을 열어 "같은 테스트셋"으로 재현 검증을 할 수 있음.
np.savez("test_data.npz", X_test=X_test, y_test=y_test, acc_inproc=acc_inproc)
print("[데이터] 테스트셋 저장 완료: test_data.npz")
```

```
if __name__ == "__main__":
    t0 = time.time()                                # 시작 시각 기록
    main()
    print(f"\n총 소요 시간: {time.time() - t0:.2f}s")    # 끝난 시각과의 차이로 전체 소요 시간 출력
```

command python train_and_pickle.py

follow_01_setup_train.png



pickle 복원 + predict + 비교

base-sklearn/predict_from_pickle.py

pickle로 저장된 model.pkl을 다시 불러와 predict한다.

이 스크립트는 model을 다시 학습하지 않는다.

train_and_pickle.py와는 완전히 별도의 python 프로세스로 쓰므로, 메모리에 있던 model 변수는 이미 사라진 상태다.

RandomForestClassifier(...).fit()을 단 한 번도 호출하지 않고,

pickle.load()만으로 학습된 model 전체(트리 구조·가중치)를 그대로 복원해 predict를 실행할 수 있다.

```
import pickle                                # model.pkl을 역직렬화(복원)하기 위해 필요
import time                                  # predict 소요 시간을 재기 위해서만 사용
import numpy as np                           # test_data.npz 로드용

def main():
    model_path = "model.pkl"

    # 1) 테스트 데이터 로드
    data = np.load("test_data.npz")          # train_and_pickle.py가 저장해둔 .npz 파일 열기
    X_test = data["X_test"]                  # 저장할 때 쓴 이름("X_test")으로 각 배열을 꺼냄
    y_test = data["y_test"]
    acc_inproc = float(data["acc_inproc"])    # 이미 계산해둔 "정답" 정확도 (numpy 스칼라 → 파이썬 float로 변환)

    # 2) pickle에서 model 복원
    # rb(read binary) 모드로 열고, pickle.load()가 트리 구조·가중치를 그대로 되살린다.
    with open(model_path, "rb") as f:        # "rb": 바이너리 읽기 모드
        model = pickle.load(f)              # 파일의 바이트를 읽어 train_and_pickle.py 때와 동일한 model 객체로 복원
    print(f"[pickle] 로드 완료: {model_path} ({type(model).__name__})"    # "RandomForestClassifier" 출력

    # 3) predict 실행 — 복원된 model로 바로 예측 가능(재학습 불필요)
    t0 = time.time()                          # 예측 시작 시각
    pred = model.predict(X_test)              # model.fit()을 다시 부르지 않고, 복원된 model로 바로 predict
    latency = time.time() - t0                # 예측에 걸린 시간(초)
    acc_pickle = (pred == y_test).mean()      # 복원된 model이 낸 예측의 정확도
    print(f"\n[pickle 복원 모델] 테스트 정확도: {acc_pickle:.4f} ({(pred == y_test).sum()}/{len(y_test)})")
    print(f"[pickle 복원 모델] predict 소요 시간 ({len(X_test)}개 샘플): {latency * 1000:.3f} ms")
    # latency는 초 단위라 *1000으로 밀리초(ms) 변환해서 출력
```



```

import numpy as np
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

def main():
    # 1) 데이터 준비 — Iris 데이터셋(150 샘플, 4 특성, 3 클래스)
    X, y = load_iris(return_X_y=True) # X: 4개 특성, y: 품종 레이블(0/1/2)
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=0, stratify=y
    )

    # 2) 학습
    model = RandomForestClassifier(random_state=0) # 모델 생성
    model.fit(X_train, y_train) # 실제 학습 실행

    # 3) 학습 직후 정확도 확인 — app.py가 서빙할 model과 같은 model
    pred = model.predict(X_test)
    acc = (pred == y_test).mean()

    # 이 acc 값이 나중에 FastAPI 서버(app.py)가 같은 입력으로 예측했을 때와 일치해야 할 기준값
    print(f"[학습 직후] 테스트 정확도: {acc:.4f} ({(pred == y_test).sum()}/{len(y_test)})")

    # 4) pickle로 저장 — app.py가 lifespan에서 그대로 로드
    model_path = "model.pkl"
    with open(model_path, "wb") as f:
        pickle.dump(model, f) # model 전체(트리 구조·가중치)를 파일로 직렬화
    print(f"[pickle] 저장 완료: {model_path}")

    # 5) curl 테스트가 재사용할 원본(cm 단위) 테스트셋도 저장
    # X_test를 float32로 캐스팅해서 저장 - FastAPI가 JSON으로 받을 입력값과 타입을 맞춰두기 위함
    # 여기 fastapi 버전은 실제 HTTP 요청 데이터 타입에 맞춰 float32로 저장하는 차이가 있음
    np.savez("test_data.npz", X_test=X_test.astype(np.float32), y_test=y_test, acc=acc)
    print("[데이터] 테스트셋 저장 완료: test_data.npz")

if __name__ == "__main__":
    t0 = time.time()
    main()
    print(f"\n총 소요 시간: {time.time() - t0:.2f}s")

```

command python train_and_pickle.py

[follow_01_setup_train.png](#)



```

[학습 직후] 테스트 정확도: 0.9333 (28/30)
[pickle] 저장 완료: model.pkl
[데이터] 테스트셋 저장 완료: test_data.npz

총 소요 시간: 0.04s

```

FastAPI 서버 기동 + /health 확인

FastAPI

파이썬으로 HTTP API 서버를 만들 때 쓰는 웹 프레임워크다.

"이 주소로 요청이 오면 이 파이썬 함수를 실행해서 그 결과를 돌려준다"는 걸 짧은 코드로 연결해주는 도구다.

@app.get("/health")처럼 함수 위에 붙은 줄을 데코레이터라고 하는데,

이게 "127.0.0.1:8331/health로 GET 요청이 들어오면 바로 아래 health() 함수를 실행해라"는 뜻이다.

@app.post("/predict")도 마찬가지로 "/predict로 POST 요청이 오면 predict() 함수를 실행해라"는 연결이다.

요청 검증

"이 함수는 이런 모양의 데이터를 받는다"고 미리 선언해두면,

응답 변환

lifespan

app.py

클라이언트가 잘못된 타입을 보냈을 때 FastAPI가 알아서 걸러내고 422 에러를 돌려준다.
개발자가 직접 if문으로 검증 코드를 짤 필요가 없다.
함수가 파이썬 객체(PredictResponse(...))를 그냥 return하면, FastAPI가 알아서 JSON 문자열로 바꿔서 응답으로 내보내 준다.
서버가 켜질 때 딱 한 번, 꺼질 때 딱 한 번 실행할 코드를 지정해두는 자리다.

"이런 요청엔 이렇게 응답한다"는 설계도일 뿐이고,
그 설계를 실행해서 포트를 열고 요청을 받는 건 uvicorn이라는 별도 프로그램이다.
python app.py가 아니라 uvicorn app:app으로 먹운다.
app.py엔 없고 uvicorn 명령이 그 역할을 한다.

fastapi-sklearn/train_and_pickle.py

```
import pickle
from contextlib import asynccontextmanager # lifespan을 async context manager로 정의하기 위한 데코레이터
from time import perf_counter             # predict 소요 시간 측정용 (time.time()보다 정밀한 성능 측정 전용 함수)
import numpy as np
import sklearn
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

IRIS_CLASS_NAMES = ["setosa", "versicolor", "virginica"] # 예측 인덱스(0/1/2)를 사람이 읽을 품종명으로 변환
# 서버 프로세스 전역에서 모델을 들고 있을 상태 저장소.
# 요청 핸들러(health/predict)가 여기서 꺼내 쓰지, 매번 새로 만들지 않는다.
ml_state: dict = {}

@asynccontextmanager
async def lifespan(app: FastAPI):
    # startup: 서버가 시작될 때 딱 1회만 실행되는 블록
    with open("model.pkl", "rb") as f:
        model = pickle.load(f)
    ml_state["model"] = model
    print(f"[startup] pickle 로드 완료: model.pkl ({type(model).__name__})")
    print(f"[startup] scikit-learn 버전: {sklearn.__version__}")
    print(f"[startup] n_estimators={model.n_estimators}, classes_={list(model.classes_)}")

    yield # 이 지점부터 서버가 요청을 받기 시작. yield 이전이 startup, 이후가 shutdown
    # shutdown: 서버가 종료될 때 실행되는 블록.

    ml_state.clear()

# lifespan=lifespan을 넘겨야 위에서 정의한 시작/종료 로직이 실제로 연결된다.
app = FastAPI(title="Iris scikit-learn pickle API", lifespan=lifespan)

# 요청 바디 스키마 — pydantic이 자동으로 타입 검증(잘못된 타입이면 422 에러)해준다.
class IrisFeatures(BaseModel):
    sepal_length_cm: float = Field(..., ge=0, description="꽃받침 길이(cm)")
    sepal_width_cm: float = Field(..., ge=0, description="꽃받침 너비(cm)")
    petal_length_cm: float = Field(..., ge=0, description="꽃잎 길이(cm)")
    petal_width_cm: float = Field(..., ge=0, description="꽃잎 너비(cm)")
    # ge=0: greater-or-equal 0, 음수 입력을 자동으로 거부. ...(Ellipsis)는 "필수값, 기본값 없음"이라는 뜻

# 응답 바디 스키마 — 이 형태로 응답이 자동 직렬화(JSON)된다.
class PredictResponse(BaseModel):
    predicted_class: str
    probabilities: dict
    latency_ms: float

@app.get("/health")
```

```

def health():
    # 혹시라도 lifespan이 실패해 모델이 없는 상태면 503(Service Unavailable)로 응답
    if "model" not in ml_state:
        raise HTTPException(status_code=503, detail="model not loaded")
    model = ml_state["model"]
    return {
        "status": "ok",
        "scikit_learn_version": sklearn.__version__,
        "model_type": type(model).__name__,
        "n_estimators": model.n_estimators,
    }

@app.post("/predict", response_model=PredictResponse)
def predict(features: IrisFeatures):
    # RandomForestClassifier는 학습 때 쓴 원본 cm 단위 값을 그대로 받는다.
    # PyTorch/Keras 예시처럼 별도 표준화 scaler를 재현할 필요가 없다는 점이 차이.
    raw = np.array(
        [[
            features.sepal_length_cm,
            features.sepal_width_cm,
            features.petal_length_cm,
            features.petal_width_cm,
        ]]
    )
    model = ml_state["model"] # startup에서 이미 로드해둔 그 객체를 재사용(재로딩 없음)
    t0 = perf_counter()
    probs = model.predict_proba(raw)[0] # 클래스별 확률 반환 (예: [0.98, 0.02, 0.0])
    latency_ms = (perf_counter() - t0) * 1000
    pred_idx = int(probs.argmax()) # 확률이 가장 높은 클래스의 인덱스
    return PredictResponse(
        predicted_class=IRIS_CLASS_NAMES[pred_idx],
        probabilities={name: round(float(p), 4) for name, p in zip(IRIS_CLASS_NAMES, probs)},
        latency_ms=round(latency_ms, 4),
    )

```

command `uvicorn app:app --host 127.0.0.1 --port 8331 > server.log 2>&1 &`

서버를 백그라운드(&)로 기동.

app:app은 "app.py 파일 안의 app이라는 FastAPI 인스턴스"를 가리킴.

> server.log 2>&1: 표준출력(1)과 표준에러(2)를 모두 server.log 파일로 리다이렉트해서,
나중에 startup 로그가 몇 번 찍혔는지 확인할 수 있게 함

```

for i in $(seq 1 20); do
    if curl -s -o /dev/null "http://127.0.0.1:8331/health"; then break; fi
    sleep 0.5
done

```

서버가 완전히 뜰 때까지 대기하는 폴링 루프. 0.5초 간격으로 최대 20번(=최대 10초) /health를 찔러보다가
응답이 오면(curl이 성공하면) 즉시 break.

```

-s # 진행 상황 표시 생략
-o /dev/null # 응답 본문은 버리고 성공/실패만 확인

```

`curl -s "http://127.0.0.1:8331/health" | command python -m json.tool`

서버가 뜬 걸 확인했으니 실제 /health 응답을 받아서 json.tool로 예쁘게 들여쓰기해서 출력

`cat server.log`

백그라운드로 띄운 서버의 로그 파일을 열어 [startup] 로그가 정확히 한 줄만 찍혔는지 눈으로 확인

[follow_02_serve_health.png](#)

```

--- GET /health ---
{
  "status": "ok",
  "scikit_learn_version": "1.9.0",
  "model_type": "RandomForestClassifier",
  "n_estimators": 100
}

--- server.log (startup 로그 확인) ---
INFO: Started server process [26778]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8331 (Press CTRL+C to quit)
[startup] pickle 로드 완료: model.pkl (RandomForestClassifier)
[startup] scikit-learn 버전: 1.9.0
[startup] n_estimators=100, classes_=[np.int64(0), np.int64(1), np.int64(2)]
INFO: 127.0.0.1:55251 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:55252 - "GET /health HTTP/1.1" 200 OK

```

POST /predict 3회 호출 + 서버 종료

server.log의 [startup] 로그가 정확히 1번만 찍히는지(health 호출 여러 번에도 늘지 않음), 세 요청의 predicted_class가 각각 setosa/versicolor/virginica와 정확히 일치하는지.

```
curl -s -X POST "http://127.0.0.1:8331/predict" -H "Content-Type: application/json" -d '{"sepal_length_cm": 5.1, "sepal_width_cm": 3.5, "petal_length_cm": 1.4, "petal_width_cm": 0.2}' | command python -m json.tool
```

```
curl -s -X POST "http://127.0.0.1:8331/predict" -H "Content-Type: application/json" -d '{"sepal_length_cm": 7.0, "sepal_width_cm": 3.2, "petal_length_cm": 4.7, "petal_width_cm": 1.4}' | command python -m json.tool
```

```
curl -s -X POST "http://127.0.0.1:8331/predict" -H "Content-Type: application/json" -d '{"sepal_length_cm": 6.3, "sepal_width_cm": 3.3, "petal_length_cm": 6.0, "petal_width_cm": 2.5}' | command python -m json.tool
```

확인이 끝나면 서버 종료

```
lsof -ti:8331 | xargs kill -9
```

[follow_03_predict_shutdown.png](#)

```

--- POST /predict (setosa 실측치: 5.1,3.5,1.4,0.2) ---
predicted_class=setosa probabilities={'setosa': 1.0, 'versicolor': 0.0, 'virginica': 0.0} latency_ms=3.1437
--- POST /predict (versicolor 실측치: 7.0,3.2,4.7,1.4) ---
predicted_class=versicolor probabilities={'setosa': 0.0, 'versicolor': 1.0, 'virginica': 0.0} latency_ms=1.5688
--- POST /predict (virginica 실측치: 6.3,3.3,6.0,2.5) ---
predicted_class=virginica probabilities={'setosa': 0.0, 'versicolor': 0.02, 'virginica': 0.98} latency_ms=1.5309

=== 서버 종료 ===
서버 종료 완료

```

PyTorch

base-pytorch

[base-pytorchWfollowup.md](#) 순서대로 진행

IrisNet(PyTorch nn.Module)을 학습해 torch.jit.script로 TorchScript(.pt)로 컴파일한 뒤, IrisNet 클래스 정의를 어디에도 import하지 않는 완전히 새로운 프로세스에서 그 .pt 파일만으로 추론해본다.

"모델 클래스 코드를 몰라도, 컴파일된 아티팩트 하나만으로 학습 때와 똑같은 정확도를 재현할 수 있는가?"

클래스와 상속(Class Inheritance)

nn.Module을 상속해서 모델을 정의한다는 객체지향 개념

Linear (레이어)

입력값들에 가중치를 곱하고 더하는 단계다.

입력 여러 개를 받아서 (가중치 × 입력) + 편향 형태로 계산해 다음 층으로 넘길 값을 만든다.

이 가중치와 편향이 신경망이 "학습"으로 알아내는 숫자들이고, 처음엔 무작위 값에서 시작한다.

활성화 함수(ReLU 등)

Linear 계산만 계속 쌓으면 아무리 층을 늘려도 결과적으로 하나의 단순한 직선 관계와 다를 게 없다.

순전파(forward)	중간에 비선형 함수를 끼워 넣어야 복잡하고 굽은 패턴을 표현할 수 있게 된다. ReLU는 그중 가장 단순한 형태로, 음수 값은 0으로 막고 양수 값은 그대로 통과시키는 방식이다. 입력이 레이어들을 순서대로 통과하면서 최종 출력(예측값)이 나오기까지의 전체 계산 과정이다.
손실함수(Loss)	"앞으로 흘러간다"고 해서 순전파다. 신경망이 내놓은 예측값과 실제 정답을 비교해서 "얼마나 틀렸는지"를 숫자 하나로 나타내는 함수다. 학습이라는 것 자체가 결국 이 손실값을 최대한 작게 만드는 작업이다.
역전파(backward) + 경사하강법(옵티마이저)	손실값이 나오면, 신경망 안의 각 가중치를 아주 조금 늘리거나 줄였을 때 손실이 어느 방향으로 얼마나 변하는지를 계산한다. 이 계산을 출력 쪽에서 입력 쪽으로 거슬러 올라가며 하기 때문에 "역"전파다. 그렇게 구한 방향을 따라 실제로 가중치를 조금씩 수정하는 단계가 경사하강법이고, 이걸 자동화·개선했서 실행하는 알고리즘이 옵티마이저(Adam, SGD 등)다.
epoch	이 다섯 단계를 한 번 도는 게 학습 1회(1 epoch 내의 1 step)이고, 이걸 수백~수천 번 반복하면서 가중치가 점점 정답을 잘 맞추는 방향으로 수렴해가는 것이 신경망 학습의 전체 그림이다.

```
cd base-pytorch
uv venv --python 3.11 --clear .venv
source .venv/bin/activate
uv pip install --quiet torch scikit-learn numpy
```

모델 학습 + TorchScript 컴파일

[base-pytorch/train_and_script.py](#)

PyTorch로 붓꽃(Iris) 분류 모델을 학습하고 TorchScript로 컴파일해 저장한다.

`torch.jit.script(model)`은 IrisNet의 Python 코드(forward 메서드)를 분석해서
자체적으로 실행 가능한 그래프 표현(ScriptModule)으로 컴파일한다.

저장된 .pt 파일은 그래프 담고 있어서, 2단계는 IrisNet 클래스 정의를 아예 import하지 않고도 로드해서 실행할 수 있다.
ONNX가 "프레임워크 간" 이식성을 위한 것이라면, TorchScript는 "PyTorch 생태계 안에서"
Python 인터프리터 자체에 대한 의존을 끊기 위한 것이다

```
import time                                     # 전체 실행 시간 측정용
import numpy as np
import torch
import torch.nn as nn                          # 신경망 레이어(Linear, ReLU 등)와 손실 함수를 담은 모듈
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler # 신경망 입력을 표준화(평균0, 분산1)하기 위한 도구
```

```
class IrisNet(nn.Module):
    # 4개 특성(꽃받침/꽃잎 길이·너비) -> 3개 클래스(붓꽃 품종)로 분류하는 작은 MLP.
    # 구조: 입력4 → Linear(4,16) → ReLU → Linear(16,8) → ReLU → Linear(8,3) → 출력3(로짓)
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(4, 16),
            nn.ReLU(),
            nn.Linear(16, 8),
            nn.ReLU(),
            nn.Linear(8, 3),
        )
        # nn.Module의 초기화 로직을 먼저 실행 (파라미터 추적 등을 위해 필수)
        # 레이어들을 순서대로 실행하는 컨테이너
        # 입력 4개 특성 → 16개 뉴런으로 선형 변환
        # 비선형 활성화 함수 (음수는 0으로, 양수는 그대로)
        # 마지막은 클래스 3개에 대응하는 값(로짓) 출력, 활성화 함수 없음
    def forward(self, x):
        # 이 모델에 입력이 실제로 흘러가는 경로를 정의
        return self.net(x)
```

```
def main():
    torch.manual_seed(0) # 가중치 초기화 난수 고정 — 재현성 확보
    # 1) 데이터 준비 — Iris 데이터셋을 8:2로 분할 후 StandardScaler로 표준화
```

```

# (신경망은 스케일에 민감하므로 scikit-learn과 달리 표준화가 필수)
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0, stratify=y
)
scaler = StandardScaler().fit(X_train)
X_train = scaler.transform(X_train).astype(np.float32)
X_test = scaler.transform(X_test).astype(np.float32)

# 학습 데이터의 평균·표준편차만 기준으로 삼음
# 표준화 후 float32로 변환(PyTorch 기본 텐서 타입)
# test도 같은 scaler 기준으로 변환(재학습 없이 통계만 적용)

X_train_t = torch.from_numpy(X_train)
y_train_t = torch.from_numpy(y_train).long()

# numpy 배열을 PyTorch 텐서로 변환
# 분류 레이블은 정수형이어야 CrossEntropyLoss가 받아들임

# 2) 학습 — 평범한 PyTorch eager mode 학습 루프(직접 backward/step 호출)
model = IrisNet()
optimizer = torch.optim.Adam(model.parameters(), lr=0.05)
loss_fn = nn.CrossEntropyLoss()
model.train()
for epoch in range(200):
    optimizer.zero_grad()
    logits = model(X_train_t)
    loss = loss_fn(logits, y_train_t)
    loss.backward()
    optimizer.step()
    if (epoch + 1) % 50 == 0:
        print(f" epoch {epoch + 1}>3}/200 loss={loss.item():.4f}")

# Adam 옵티마이저
# 다중 클래스 분류용 손실 함수(내부에서 softmax까지 처리)
# 학습 모드로 전환
# 이전 스텝의 gradient 초기화
# forward — 모델에 입력을 흘려 예측값(로짓) 계산
# 역전파로 gradient 계산(autograd)
# 계산된 gradient로 가중치 업데이트
# 50 epoch마다 한 번씩만 진행 상황 출력

# 3) 학습 정확도 확인 (eager mode PyTorch 자체 추론) — TorchScript와 비교할 기준값
model.eval()
with torch.no_grad():
    X_test_t = torch.from_numpy(X_test)
    pred_eager = model(X_test_t).argmax(dim=1).numpy()
acc_eager = (pred_eager == y_test).mean()
print(f"[Eager PyTorch] 테스트 정확도: {acc_eager:.4f} ({(pred_eager == y_test).sum()}/{len(y_test)})")

# 평가 모드로 전환(이 모델엔 드롭아웃/배치노름이 없지만 관례상 항상 호출)
# 추론 시엔 gradient 계산이 필요 없으므로 꺼서 메모리·속도 절약
# 3개 클래스 중 가장 높은 로짓의 인덱스를 예측값으로

# 4) TorchScript로 컴파일
# torch.jit.script는 forward()의 Python 코드를 "직접 분석(파싱)"해서 정적 그래프(ScriptModule)로 변환한다.
# 더미 입력을 실행해서 그래프를 추적하는 trace 방식과 달리,
# script 방식은 조건문·반복문이 있어도 실제 제어 흐름 구조 자체를 그대로 컴파일한다.
scripted = torch.jit.script(model)
print(f"[TorchScript] 컴파일된 그래프(code):")
print(scripted.code)
pt_path = "iris_model.pt"
scripted.save(pt_path)
print(f"[TorchScript] 저장 완료: {pt_path}")

# 컴파일된 결과를 사람이 읽을 수 있는 형태(의사 파이썬 코드)로 출력
# ScriptModule을 파일로 저장 — 그래프+가중치가 통째로 담김

# 5) TorchScript 모델로도 같은 입력에 같은 결과가 나오는지 같은 프로세스에서 우선 확인
with torch.no_grad():
    pred_script = scripted(X_test_t).argmax(dim=1).numpy()
acc_script = (pred_script == y_test).mean()

# 원래 model이 아니라 컴파일된 scripted로 예측

# 이 값이 위 acc_eager와 같아야, 컴파일 과정에서 모델 동작이 안 바뀌었다는 걸 확인할 수 있음
print(f"[TorchScript(같은 프로세스)] 테스트 정확도: {acc_script:.4f}")

# 6) 다음 단계(infer_torchscript.py, 별도 프로세스)에서 재사용할 테스트 데이터·정확도 저장
np.savez("test_data.npz", X_test=X_test, y_test=y_test, acc_eager=acc_eager)
print("[데이터] 테스트셋 저장 완료: test_data.npz")

if __name__ == "__main__":
    t0 = time.time()

```

```
main()
print(f"\n총 소요 시간: {time.time() - t0:.2f}s")
```

command python train_and_script.py

[follow_01_setup_train.png](#)

```
torch 2.14.0
scikit-learn 1.9.0

epoch 50/200 loss=0.0423
epoch 100/200 loss=0.0361
epoch 150/200 loss=0.0324
epoch 200/200 loss=0.0053
[Eager PyTorch] 테스트 정확도 : 0.9667 (29/30)
/Users/manure/_work/re_scala/output/_통합개념_실습/PYTORCH/base-pytorch/.venv/lib/python3.11/site-packages/torch/jit/_script.py:1491: FutureWarning: `torch.jit.script` is deprecated. Please switch to `torch.compile` or `torch.export`.
  warnings.warn(
[TorchScript] 컴파일된 그래프 (code):
def forward(self,
  x: Tensor) -> Tensor:
  net = self.net
  return (net).forward(x, )

[TorchScript] 저장 완료 : iris_model.pt
[TorchScript(같은 프로세스)] 테스트 정확도 : 0.9667
[데이터] 테스트셋 저장 완료 : test_data.npz

총 소요 시간 : 0.62s
```

TorchScript(.pt)만으로 추론 + 비교

[base-pytorch/infer_torchscript.py](#)

저장된 TorchScript(.pt) 모델을 IrisNet 클래스 정의 없이 로드해 추론한다.

이 스크립트는 IrisNet 클래스를 import하지 않는다.

torch.jit.load()만으로 .pt 파일 안에 저장된 그래프와 가중치를 그대로 복원해 실행할 수 있다는 것이 TorchScript가 해결하는 문제다. 모델 코드를 몰라도, 컴파일된 아티팩트 하나만 있으면 실행할 수 있다.

콘솔에 FutureWarning(torch.jit.script is deprecated...) 경고가 뜰 수 있는데 결과엔 영향 없으니 무시하고 진행

저장된 TorchScript(.pt) 모델을 IrisNet 클래스 정의 없이 로드해 추론한다.

```
import time
```

```
import numpy as np
```

```
import torch
```

```
def main():
```

```
    pt_path = "iris_model.pt"
```

```
    # 1) 테스트 데이터 로드 (1단계 train_and_script.py가 저장해둔 것)
```

```
    data = np.load("test_data.npz")
```

```
    X_test = data["X_test"].astype(np.float32)
```

```
    y_test = data["y_test"]
```

```
    acc_eager = float(data["acc_eager"])
```

1단계에서 저장한 eager 모드 정확도 (비교 기준값)

```
    # 2) TorchScript 모델 로드 — IrisNet 클래스 정의가 이 프로세스 어디에도 없다
```

```
    torch.jit.set_fusion_strategy([("STATIC", 0)])
```

재현성을 위해 JIT 퓨전 최적화 끄

```
    scripted = torch.jit.load(pt_path)
```

```
    scripted.eval()
```

```
    print(f"[TorchScript] 로드 완료: {pt_path}")
```

```
    print(f"[TorchScript] 타입: {type(scripted)}")
```

```
    # 3) 추론 실행 — 일반 nn.Module과 동일하게 호출 가능
```

```
    X_test_t = torch.from_numpy(X_test)
```

```
    t0 = time.time()
```

```
    with torch.no_grad():
```

```

logits = scripted(X_test_t)
latency = time.time() - t0
pred_script = logits.argmax(dim=1).numpy()
acc_script = (pred_script == y_test).mean()
print(f"\n[TorchScript] 테스트 정확도: {acc_script:.4f} ({(pred_script == y_test).sum()}/{len(y_test)})")
print(f"[TorchScript] 추론 소요 시간 ({len(X_test)}개 샘플): {latency * 1000:.3f} ms")

# 4) eager mode(1단계) 결과와 일치하는지 검증
print(f"\n[비교] Eager PyTorch 테스트 정확도: {acc_eager:.4f}")
print(f"[비교] TorchScript 테스트 정확도 : {acc_script:.4f}")
if abs(acc_eager - acc_script) < 1e-6:
    print("[비교] 결과 일치 ✅ — IrisNet 클래스 없이도 컴파일된 .pt 파일만으로 동일하게 재현했다.")
else:
    print("[비교] 결과 불일치 ⚠️")

if __name__ == "__main__":
    main()

```

command python infer_torchscript.py

follow_02_infer_compare.png

```

/Users/manure/_work/re_scala/output/_통합 개념 실습 /PYTORCH/base-pytorch/.venv/lib/python3.11/site-packages/torch/jit/_serialization.py:176: FutureWarning: `torch.jit.load` is deprecated. Please switch to `torch.export`.
  warnings.warn(
[TorchScript] 로드 완료 : iris_model.pt
[TorchScript] 타입 : <class 'torch.jit._script.RecursiveScriptModule'>

[TorchScript] 테스트 정확도 : 0.9667 (29/30)
[TorchScript] 추론 소요 시간 (30개 샘플) : 5.326 ms

[비교] Eager PyTorch 테스트 정확도 : 0.9667
[비교] TorchScript 테스트 정확도 : 0.9667
[비교] 결과 일치 ✅ — IrisNet 클래스 없이도 컴파일된 .pt 파일만으로 동일하게 재현했다.

```

fastapi-pytorch

TorchScript 서빙 + 워커 수 성능 비교

[fastapi-pytorch](#) [followup.md](#) 순서대로 진행

TorchScript(.pt)로 컴파일한 모델을 FastAPI로 서빙하면서, 코드를 한 글자도 바꾸지 않고

uvicorn 워커 수만 1개 → 4개로 늘렸을 때 처리량과 지연시간이 어떻게 달라지는지 실제 부하 테스트로 확인한다.

"같은 서빙 코드인데, 프로세스(워커) 수만 늘리면 정말로 처리량이 배로 늘어나는가?"

프로세스(Process) vs 스레드(Thread)

프로세스는 OS가 자원을 할당하는 독립적인 실행 단위다.

각 프로세스는 자기만의 메모리 공간, 자기만의 Python 인터프리터를 따로 가진다.

스레드는 하나의 프로세스 "안에서" 실행되는 더 작은 단위로,

같은 프로세스 안의 여러 스레드는 메모리 공간을 공유한다.

프로세스를 새로 띄우는 건 무겁고(메모리 복사, 인터프리터 초기화),

스레드를 새로 띄우는 건 상대적으로 가볍다.

GIL(Global Interpreter Lock)

CPython의 구현 특성이다.

한 프로세스 안에서는 몇 개의 스레드를 만들든 상관없이,

그 순간 실제로 파이썬 바이트코드를 실행할 수 있는 스레드는 딱 하나뿐이다.

CPU 코어가 8개짜리 서버라도, 파이썬은 8개 코어를 동시에 다 못 쓴다.

프로세스를 여러 개 띄우면 그만큼 진짜로 여러 코어를 동시에 쓴다.

동시성(Concurrency)

동시성은 "여러 요청이 동시에 진행 중인 상태"를 말하는 거지,

그게 다 동시에 처리되고 있다는 뜻은 아니다.

uvicorn —workers

이 옵션을 주면 uvicorn이 마스터 프로세스 하나 아래에 지정한 개수만큼

독립된 워커 프로세스를 fork해서 띄운다.

각 워커는 앱 코드를 통째로 복사해서 자기만의 메모리·인터프리터·GIL을 가진 채

같은 포트를 공유해서 리스닝하고, OS가 들어오는 연결을 워커들에게 나눠준다.

cd fastapi-pytorch

```
uv venv --python 3.11 --clear .venv
source .venv/bin/activate
uv pip install --quiet torch scikit-learn numpy fastapi "uvicorn[standard]"
```

[fastapi-pytorch/train_and_script.py](#)

PyTorch로 붓꽃(Iris) 분류 모델을 학습하고 TorchScript로 컴파일해 저장한다.

base-pytorch와 같은 모델·컴파일 방식(torch.jit.script)을 쓰되, 이 폴더는 FastAPI 서버까지 포함하므로 학습 때 쓴 StandardScaler의 mean_/scale_도 scaler.npz로 함께 저장한다.

app.py(서빙 쪽)는 sklearn을 아예 import하지 않고 이 두 배열만으로 표준화를 재현한다.

학습 시점 전처리와 서빙 시점 전처리가 어긋나면 정확도가 "조용히" 망가진다는, 실제 서버에서 혼한 합정을 그대로 보여주기 위한 구성이니 학생들에게 이 부분을 꼭 짚어줄 것.

```
import time
import numpy as np
import torch
import torch.nn as nn
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

class IrisNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(4, 16),
            nn.ReLU(),
            nn.Linear(16, 8),
            nn.ReLU(),
            nn.Linear(8, 3),
        )
    def forward(self, x):
        return self.net(x)

def main():
    torch.manual_seed(0)

    # 1) 데이터 준비 — Iris 데이터셋을 학습/테스트로 분리 후 표준화
    X, y = load_iris(return_X_y=True)
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=0, stratify=y
    )
    scaler = StandardScaler().fit(X_train)
    X_train_scaled = scaler.transform(X_train).astype(np.float32)
    X_test_scaled = scaler.transform(X_test).astype(np.float32)
    X_train_t = torch.from_numpy(X_train_scaled)
    y_train_t = torch.from_numpy(y_train).long()

    # 2) 학습 — 평범한 PyTorch eager mode 학습 루프
    model = IrisNet()
    optimizer = torch.optim.Adam(model.parameters(), lr=0.05)
    loss_fn = nn.CrossEntropyLoss()
    model.train()
    for epoch in range(200):
        optimizer.zero_grad()
        logits = model(X_train_t)
        loss = loss_fn(logits, y_train_t)
        loss.backward()
        optimizer.step()
        if (epoch + 1) % 50 == 0:
            print(f" epoch {epoch + 1:>3}/200 loss={loss.item():.4f}")
```

```

# 3) 학습 정확도 확인 (eager mode)
model.eval()
with torch.no_grad():
    X_test_t = torch.from_numpy(X_test_scaled)
    pred_eager = model(X_test_t).argmax(dim=1).numpy()
acc_eager = (pred_eager == y_test).mean()
print(f"[Eager PyTorch] 테스트 정확도: {acc_eager:.4f} ({(pred_eager == y_test).sum()}/{len(y_test)})")

# 4) TorchScript로 컴파일 — forward()의 Python 코드를 직접 분석해서 정적 그래프로 변환
scripted = torch.jit.script(model)
pt_path = "iris_model.pt"
scripted.save(pt_path)
print(f"[TorchScript] 저장 완료: {pt_path}")

# 5) base-pytorch와의 차이점: app.py가 학습 때와 동일하게 표준화를 재현할 수
# 있도록 StandardScaler의 mean_/scale_을 그대로 저장 — sklearn 없이도 복원 가능
np.savez("scaler.npz", mean=scaler.mean_.astype(np.float32), scale=scaler.scale_.astype(np.float32))
print("[Scaler] 저장 완료: scaler.npz (mean, scale)")

# 6) run.sh/curl 테스트가 재사용할, 표준화 전 원본(cm 단위) 테스트 데이터
X_test_raw = X_test.astype(np.float32)
np.savez("test_data.npz", X_test_raw=X_test_raw, y_test=y_test, acc_eager=acc_eager)
print("[데이터] 테스트셋 저장 완료: test_data.npz")

```

```

if __name__ == "__main__":
    t0 = time.time()
    main()
    print(f"\n총 소요 시간: {time.time() - t0:.2f}s")

```

command python train_and_script.py

[follow_01_setup_train.png](#)

```

Using CPython 3.11.15
Creating virtual environment at: .venv
Activate with: source .venv/bin/activate
torch 2.14.0
fastapi 0.141.1
uvicorn 0.52.4

epoch 50/200 loss=0.0423
epoch 100/200 loss=0.0361
epoch 150/200 loss=0.0324
epoch 200/200 loss=0.0053
[Eager PyTorch] 테스트 정확도 : 0.9667 (29/30)
/Users/manure/_work/re_scala/output/_통합개념_실습/PYTORCH/fastapi-pytorch/.venv/lib/python3.11/site-packages/torch/jit/_script.py:1491: FutureWarning: `torch.jit.script` is deprecated. Please switch to `torch.compile` or `torch.export`.
  warnings.warn(
[TorchScript] 저장 완료: iris_model.pt
[Scaler] 저장 완료: scaler.npz (mean, scale)
[데이터] 테스트셋 저장 완료: test_data.npz

총 소요 시간: 1.73s

```

FastAPI 서버 기동(1 워커) + /health + /predict 3회

[fastapi-pytorch/app.py](#)

torch.jit.load()로 불러온 TorchScript 모델을 FastAPI로 서빙한다.

base-pytorch에서 확인한 "클래스 정의 없이 추론 가능"이 서빙 상황에서도 그대로 성립함을 보여준다.

lifespan으로 모델 스케일러를 프로세스 시작 시 1회만 로드해 요청마다의 로딩 비용을 없앤다.

scikit-learn 예시의 lifespan 패턴과 동일한 설계.

```

from contextlib import asynccontextmanager
import numpy as np
import torch

```

```

from fastapi import FastAPI
from pydantic import BaseModel, Field

CLASS_NAMES = ["setosa", "versicolor", "virginica"]
ml_state: dict = {}

@asynccontextmanager
async def lifespan(app: FastAPI):
    # torch.jit.load()는 iris_model.pt 안에 저장된 연산 그래프 자체로 모델을
    # 복원한다 — IrisNet 클래스 정의가 이 프로세스에 없어도 동작한다.
    model = torch.jit.load("iris_model.pt")
    model.eval()
    torch.set_grad_enabled(False)          # 서버 프로세스 전체에서 autograd 비활성화(추론 전용이므로 불필요한 계산 제거)
    scaler = np.load("scaler.npz")
    ml_state["model"] = model
    ml_state["mean"] = scaler["mean"]
    ml_state["scale"] = scaler["scale"]
    print(f"[lifespan] TorchScript 모델 로드 완료 (torch {torch.__version__})")

    yield

    ml_state.clear()
    print("[lifespan] 모델 리소스 해제 완료")

app = FastAPI(title="Iris Classifier (TorchScript)", lifespan=lifespan)

class IrisFeatures(BaseModel):
    sepal_length_cm: float = Field(..., ge=0, le=15, description="꽃받침 길이(cm)")
    sepal_width_cm: float = Field(..., ge=0, le=15, description="꽃받침 너비(cm)")
    petal_length_cm: float = Field(..., ge=0, le=15, description="꽃잎 길이(cm)")
    petal_width_cm: float = Field(..., ge=0, le=15, description="꽃잎 너비(cm)")

class PredictResponse(BaseModel):
    predicted_class: str
    predicted_index: int
    probabilities: dict[str, float]

@app.get("/health")
def health():
    return {"status": "ok", "model_loaded": "model" in ml_state, "runtime": "torchscript"}

@app.post("/predict", response_model=PredictResponse)
def predict(features: IrisFeatures):
    raw = np.array(
        [
            features.sepal_length_cm,
            features.sepal_width_cm,
            features.petal_length_cm,
            features.petal_width_cm,
        ],
        dtype=np.float32,
    )
    # 학습 때와 동일한 표준화 공식을 numpy 연산으로 직접 재현: (x - 평균) / 표준편차
    scaled = (raw - ml_state["mean"]) / ml_state["scale"]
    x = torch.from_numpy(scaled)
    with torch.no_grad():
        logits = ml_state["model"](x)
        probs = torch.softmax(logits, dim=1)[0].numpy()
        idx = int(probs.argmax())

```

```

return PredictResponse(
    predicted_class=CLASS_NAMES[idx],
    predicted_index=idx,
    probabilities={name: float(p) for name, p in zip(CLASS_NAMES, probs)},
)

```

1) FastAPI 앱을 uvicorn(1 워커, 기본값)으로 백그라운드 기동
 # 출력은 화면에 안 찍고 server.log 파일로 리다이렉트(> ... 2>&1), 뒤의 &는 백그라운드 실행
 command uvicorn app:app --host 127.0.0.1 --port 8331 > server.log 2>&1 &

2) 서버가 완전히 뜨기까지 시간이 걸리므로, /health가 응답할 때까지 대기
 # 0.5초 간격으로 최대 20번(=10초) 재시도, 응답 오면 즉시 break
 for i in \$(seq 1 20); do
 if curl -s -o /dev/null "http://127.0.0.1:8331/health"; then break; fi
 sleep 0.5
 done

3) 헬스체크 응답 확인(status, model_loaded, runtime이 정상인지)
 curl -s "http://127.0.0.1:8331/health" | command python -m json.tool

4) setosa 실측치(5.1, 3.5, 1.4, 0.2)로 /predict 호출
 # -H: 요청 헤더에 JSON 타입 명시, -d: 요청 본문(JSON), 결과를 json.tool로 보기 좋게 출력
 curl -s -X POST "http://127.0.0.1:8331/predict" \W
 -H "Content-Type: application/json" \W
 -d '{"sepal_length_cm": 5.1, "sepal_width_cm": 3.5, "petal_length_cm": 1.4, "petal_width_cm": 0.2}' \W
 | command python -m json.tool
 # versicolor(7.0,3.2,4.7,1.4), virginica(6.3,3.3,6.0,2.5)도 같은 방식으로 확인

[follow_02_serve_predict.png](#)

```

--- GET /health ---
{
  "status": "ok",
  "model_loaded": true,
  "runtime": "torchscript"
}

--- POST /predict x3 ---
[setosa] predicted_class=setosa top_prob=1.0000
[versicolor] predicted_class=versicolor top_prob=0.9999
[virginica] predicted_class=virginica top_prob=1.0000

```

perf_test.py

부하 테스트 스크립트

[fastapi-pytorch/perf_test.py](#)

실제로 떠 있는 FastAPI + TorchScript 서버에 부하를 걸어 성능을 측정한다.
 클라이언트가 ThreadPoolExecutor를 쓰면 "클라이언트 자신의" GIL이 서버 병목과 섞여 측정된다.
 그래서 동시성 개수만큼 별도 프로세스(각자 자기만의 GIL)를 미리 띄워두고,
 각 프로세스가 자기 몫의 요청을 순차로 다 보낸 뒤 결과를 한 번에 반환하게 한다.

```

import json
import multiprocessing
import multiprocessing.pool
import statistics
import time
import urllib.request

```

```

BASE_URL = "http://127.0.0.1:8331"
MAX_CONCURRENCY = 50
REQUESTS_PER_WORKER = 40

```

```

# 테스트 대상 서버 주소(app.py가 떠 있는 곳)
# 이 실습에서 시험할 최대 동시성(=최대로 만들 프로세스 수)
# 워커(=동시성 한 단위)마다 순차로 보낼 요청 수

```

```

# ONNX/base_fastapi와 동일한 3개 실측 표본을 그대로 순환 사용
# 결과를 나란히 비교할 수 있도록 페이로드를 일치시킨다.
PAYLOADS = [
    {"sepal_length_cm": 5.1, "sepal_width_cm": 3.5, "petal_length_cm": 1.4, "petal_width_cm": 0.2}, # setosa
    {"sepal_length_cm": 7.0, "sepal_width_cm": 3.2, "petal_length_cm": 4.7, "petal_width_cm": 1.4}, # versicolor
    {"sepal_length_cm": 6.3, "sepal_width_cm": 3.3, "petal_length_cm": 6.0, "petal_width_cm": 2.5}, # virginica
]

def one_request(i: int) -> tuple[float, bool]:
    # predict에 요청 1건을 보내고 (왕복 지연시간_ms, 성공여부)를 반환한다.
    # i번째 요청은 세 표본 중 (i % 3)번째를 사용 — 요청마다 골고루 순환
    payload = json.dumps(PAYLOADS[i % len(PAYLOADS)]).encode("utf-8")
    req = urllib.request.Request(
        f"{BASE_URL}/predict",
        data=payload,
        headers={"Content-Type": "application/json"},
        method="POST",
    )
    t0 = time.perf_counter() # 요청 보내기 직전 시각(고정밀 타이머)
    try:
        with urllib.request.urlopen(req, timeout=5) as resp:
            resp.read() # 응답 본문을 끝까지 읽어야 요청이 완전히 끝난 것으로 침
            ok = resp.status == 200 # HTTP 200이면 성공
    except Exception:
        ok = False # 타임아웃/커넥션 에러 등은 전부 실패로 처리
    return (time.perf_counter() - t0) * 1000, ok # 왕복 시간을 ms 단위로 환산해서 반환

def worker_loop(n_requests: int) -> list[tuple[float, bool]]:
    # 워커 프로세스 하나가 n_requests건을 자기 프로세스 안에서 순차로 보내고,
    # (지연시간, 성공여부) 리스트를 한 번에 반환한다.
    # 부모-자식 간 IPC는 이 함수 호출 1번(제출 1번 + 반환 1번)뿐이다.
    # 이 함수 자체가 각 워커 프로세스 안에서 실행되는 코드다.
    # 리스트 하나를 한 번에 만들어서 반환하므로, 요청을 보낼 때마다 부모 프로세스와 통신(IPC)하지 않는다.
    # 통신 비용이 측정치를 왜곡하는 걸 방지.
    return [one_request(i) for i in range(n_requests)]

def percentile(sorted_values: list, p: float) -> float:
    # 이미 정렬된 리스트에서 p번째 백분위수를 인덱스 계산으로 바로 꺼내는 방식
    # (예: p=0.95, 길이 100이면 idx=95 → 상위 5%를 걸러낸 값)
    if not sorted_values:
        return float("nan")
    idx = min(len(sorted_values) - 1, int(len(sorted_values) * p)) # 배열 끝을 넘지 않도록 min으로 보정
    return sorted_values[idx]

def run_load(pool: multiprocessing.pool.Pool, concurrency: int, requests_per_worker: int) -> dict:
    # 미리 예열해둔 풀(최대 동시성 크기로 고정)에서 `concurrency`개 워커만 골라 쏜다.
    # 나머지 워커는 이번 구간에서 그냥 유휴 상태. 각 워커는 자기
    # 몫(requests_per_worker건)을 순차로 보내고 끝나면 결과를 한 번에 반환한다.
    # 총 요청 수는 concurrency * requests_per_worker
    total_requests = concurrency * requests_per_worker
    t_start = time.perf_counter()

    # pool.map: concurrency개의 워커 각각에게 requests_per_worker를 인자로 넘겨
    # worker_loop를 동시에 실행시키고, 전부 끝날 때까지 기다렸다가 결과 리스트를 모아 받는다.
    per_worker_results = pool.map(worker_loop, [requests_per_worker] * concurrency)
    wall_time = time.perf_counter() - t_start # 이 구간 전체가 걸린 실제 경과 시간
    latencies = []
    errors = 0

```

```

# 워커별 결과 리스트를 다시 한 줄짜리 리스트로 평탄화하면서 실패 건수를 센다
for worker_results in per_worker_results:
    for latency_ms, ok in worker_results:
        latencies.append(latency_ms)
        if not ok:
            errors += 1
latencies.sort()
return {
    "concurrency": concurrency,
    "total_requests": total_requests,
    "errors": errors,
    "wall_time_s": wall_time,
    "throughput_rps": total_requests / wall_time,
    "latency_mean_ms": statistics.mean(latencies),
    "latency_p50_ms": percentile(latencies, 0.50),
    "latency_p90_ms": percentile(latencies, 0.90),
    "latency_p95_ms": percentile(latencies, 0.95),
    "latency_p99_ms": percentile(latencies, 0.99),
    "latency_max_ms": latencies[-1],
}

def main():
    # MAX_CONCURRENCY(50)개의 프로세스를 담은 풀을 딱 한 번만 생성
    # 이후 모든 동시성 구간(1/5/20/50)이 이 풀을 공유해서 재사용한다.
    with multiprocessing.Pool(processes=MAX_CONCURRENCY) as pool:
        # 풀이 워커 프로세스 MAX_CONCURRENCY개를 실제로 다 띄우기까지 걸리는 spawn 비용을 여기서 한 번에 치르고,
        # 이후 구간 측정에서는 제외한다.
        print(f"[warmup] {MAX_CONCURRENCY}개 워커 예열 중...")
        run_load(pool, concurrency=MAX_CONCURRENCY, requests_per_worker=REQUESTS_PER_WORKER)

    # 결과표 머리글 출력
    print(f"\n\n{'동시성':>6} {'요청수':>6} {'실패':>4} {'처리량(req/s)':>14} {'평균(ms)':>9} {'p50(ms)':>9} {'p95(ms)':>9} {'p99(ms)':>9} {'최대(ms)':>9}")
    print("-" * 90)
    results = []

    # 동시성을 1 -> 5 -> 20 -> 50으로 점점 늘려가며 각각 측정
    for concurrency in [1, 5, 20, 50]:
        r = run_load(pool, concurrency, requests_per_worker=REQUESTS_PER_WORKER)
        results.append(r)
        print(
            f"{r['concurrency']:>6} {r['total_requests']:>6} {r['errors']:>4} "
            f"{r['throughput_rps']:>14.1f} {r['latency_mean_ms']:>9.2f} "
            f"{r['latency_p50_ms']:>9.2f} {r['latency_p95_ms']:>9.2f} "
            f"{r['latency_p99_ms']:>9.2f} {r['latency_max_ms']:>9.2f}"
        )

    # 동시성 1과 50, 양극단만 뽑아서 요약 비교
    print("\n\n[요약] 동시성 1 -> 50으로 올렸을 때:")
    r1, r50 = results[0], results[-1]
    print(f"    처리량: {r1['throughput_rps']:.1f} req/s -> {r50['throughput_rps']:.1f} req/s ({r50['throughput_rps'] / r1['throughput_rps']:.2f}배)")
    print(f"    p50 지연시간: {r1['latency_p50_ms']:.2f}ms -> {r50['latency_p50_ms']:.2f}ms")
    print(f"    p99 지연시간: {r1['latency_p99_ms']:.2f}ms -> {r50['latency_p99_ms']:.2f}ms")

if __name__ == "__main__":
    main()

```

이전 서버가 켜져 있는 상태에서

curl -s <http://127.0.0.1:8331/health>

서버 체크

command python perf_test.py

follow_03_perf_1worker.png

```
=== 성능 테스트 - 1 워커 ===
[warmup] 50개 워커 예열 중 ...
```

	동시성	요청수	실패	처리량 (req/s)	평균 (ms)	p50(ms)	p95(ms)	p99(ms)	최대 (ms)
-	1	40	0	2059.2	0.47	0.38	0.45	4.03	4.03
-	5	200	0	4898.4	0.97	0.97	1.26	1.38	1.42
-	20	800	0	4789.5	4.09	4.11	5.67	6.50	7.46
-	50	2000	0	4510.6	10.88	10.40	14.56	28.92	32.45

```
[요약] 동시성 1 -> 50으로 올렸을 때 :
처리량 : 2059.2 req/s -> 4510.6 req/s (2.19배)
p50 지연시간 : 0.38ms -> 10.40ms
p99 지연시간 : 4.03ms -> 28.92ms

--- 1 워커 서버 종료 ---
종료 완료
```

4 워커로 재기동 + 같은 성능 테스트 + 서버 종료

app.py 코드는 한 글자도 안 바꾸고 --workers 4 플래그만 추가해서 재기동

(워커마다 lifespan이 독립적으로 다시 실행되어 4개 프로세스가 각자 모델을 로드)

command uvicorn app:app --host 127.0.0.1 --port 8331 --workers 4 > server_4workers.log 2>&1 &

4 워커는 기동에 더 걸릴 수 있어 재시도 횟수를 40번(=20초)으로 늘려서 대기

```
for i in $(seq 1 40); do
  if curl -s -o /dev/null "http://127.0.0.1:8331/health"; then break; fi
  sleep 0.5
done
```

1 워커 때와 동일한 부하 테스트를 그대로 재실행

command python perf_test.py

확인이 끝나면 서버 종료

lsof -ti:8331 | xargs kill -9

follow_04_perf_4workers.png

```
=== 성능 테스트 - 4 워커 ===
[warmup] 50개 워커 예열 중 ...
```

	동시성	요청수	실패	처리량 (req/s)	평균 (ms)	p50(ms)	p95(ms)	p99(ms)	최대 (ms)
-	1	40	0	2803.7	0.34	0.33	0.40	0.44	0.44
-	5	200	0	8639.7	0.54	0.52	0.74	0.85	0.89
-	20	800	0	11503.7	1.64	1.58	2.49	3.09	3.88
-	50	2000	0	11398.8	4.23	4.13	6.16	7.06	9.09

```
[요약] 동시성 1 -> 50으로 올렸을 때 :
처리량 : 2803.7 req/s -> 11398.8 req/s (4.07배)
p50 지연시간 : 0.33ms -> 4.13ms
p99 지연시간 : 0.44ms -> 7.06ms

=== 서버 종료 ===
/private/tmp/claude-502/-Users-manure--work-re-scala/154b049b-1078-42ba-a77b-7af3be21cbc4/scratchpad/fp_node4_perf4.sh:
line 10: 41575 Killed: 9          uvicorn app:app --host 127.0.0.1 --port 8331 --workers 4 > server_4workers.log 2>
&1
서버 종료 완료
```

Keras

base-keras

base-kerasWfollowup.md 순서대로 진행

Keras 3(TensorFlow 백엔드)로 붓꽃 분류 모델을 학습해 .keras 파일로 저장한 뒤, 모델 정의 코드가 전혀 없는 완전히 새로운 python 프로세스에서 그 파일만으로 모델을 복원해 predict까지 실행해 본다.

"재학습도, 모델 구조 재정의도 없이, 저장된 .keras 파일 하나만으로 학습 직후와 똑같은 결과를 재현할 수 있는가?"

```
Tensorflow
import tensorflow as tf
W = tf.Variable(tf.random.normal([4, 16])) # 4x16 가중치를 직접 만들
b = tf.Variable(tf.zeros([16]))          # 편향도 직접 만들
```

```
Keras
import keras
layer = keras.layers.Dense(16)          # W, b를 keras가 내부에서 알아서 만들어줌
```

Keras 1~2.x, TF 1.x~2.15 시절

Keras는 원래 프랑수아 솔레(François Chollet)가 만든 독립 라이브러리였고, 초기엔 TensorFlow·Theano·CNTK 등 여러 연산 엔진 중 아무거나 골라 쓸 수 있는 "그 위에 올라타는 얇은 API"였다.

2019년 TensorFlow 2.0부터 구글이 Keras를 TF의 공식 고수준 API로 통째로 흡수해서 tensorflow.keras(=tf.keras)라는 이름으로 TF 안에 내장시켰고, 한동안은 "Keras를 쓴다"는 게 사실상 "tf.keras를 쓴다"와 같은 말이었다.

Keras 3, 2023년 이후

다시 독립 패키지로 분리됐고, TensorFlow·JAX·PyTorch 중 원하는 걸 실행 엔진(백엔드)으로 골라 쓸 수 있게 바뀌었다.

```
os.environ.setdefault("KERAS_BACKEND", "tensorflow")
```

더 이상 TensorFlow 전용이 아니라서, "이번엔 TensorFlow 엔진을 쓰겠다"고 명시적으로 지정해야 한다. 실제로 실행 로그에도 [Keras] 버전: 3.15.1, 백엔드: tensorflow라고 버전과 백엔드가 따로 찍힌다.

Keras는 "모델을 정의하고 학습시키는 운전대·계기판"(keras.Sequential, model.fit()) 같은 사용자 친화적 API)이고, TensorFlow는 그 아래서 실제 텐서 연산·자동미분·GPU 커널을 돌리는 "엔진"이다.

```
cd base-keras
uv venv --python 3.11 --clear .venv
source .venv/bin/activate
uv pip install --quiet keras tensorflow scikit-learn numpy
```

[base-keras/train_and_save.py](#)

Keras 3(TensorFlow 백엔드)로 붓꽃(Iris) 분류 모델을 학습하고 .keras 파일로 저장한다. tensorflow.keras가 아니라 독립 패키지 keras(3.x)를 쓰되, 백엔드를 KERAS_BACKEND=tensorflow로 고정한다.

학습이 끝난 모델은 model.save("iris_model.keras") 한 줄로 Keras v3 네이티브 포맷에 저장되는데, 이 파일 하나에 아키텍처+가중치+킴파일 설정이 전부 담긴다.

```
import os
# keras를 import하기 전에 반드시 설정 — import 이후엔 백엔드를 바꿀 수 없다.
```

```
os.environ.setdefault("KERAS_BACKEND", "tensorflow")
```

```
import time
import keras
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
def build_model() -> keras.Model:
```

```
# 4개 특성 -> 3개 클래스로 분류하는 작은 MLP.
```

```
# scikit-learn·PyTorch 예시와 같은 4->16->8->3 구조
```

```
# 프레임워크 저장/복원 경로를 나란히 비교할 수 있게 의도적으로 맞췄다.
```

```
return keras.Sequential([
    keras.layers.Input(shape=(4,)),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(8, activation="relu"),
    keras.layers.Dense(3),
]) # 마지막 층은 활성화 없음(로짓 그대로 출력, loss에서 처리)
```

```

def main():
    keras.utils.set_random_seed(0) # 가중치 초기화 등 모든 난수 시드 고정
    print(f"[Keras] 버전: {keras.__version__}, 백엔드: {keras.backend.backend()}")

    # 1) 데이터 준비 — 다른 프레임워크 실습과 동일한 Iris 분할
    X, y = load_iris(return_X_y=True)
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=0, stratify=y
    )
    scaler = StandardScaler().fit(X_train)
    X_train_scaled = scaler.transform(X_train).astype(np.float32)
    X_test_scaled = scaler.transform(X_test).astype(np.float32)

    # 2) 학습
    model = build_model()
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=0.05),
        # from_logits=True
        # 모델 출력이 softmax를 거치지 않으므로, loss 계산 시 여기서 softmax를 함께 적용하도록 지정(수치적으로 더 안정적)
        loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
        metrics=["accuracy"],
    )
    model.fit(X_train_scaled, y_train, epochs=200, verbose=0)

    # 3) 학습 직후(같은 프로세스) 정확도 확인 — 2단계 결과와 비교할 기준값
    pred = model.predict(X_test_scaled, verbose=0).argmax(axis=1)
    acc_inproc = float((pred == y_test).mean())
    print(f"[학습 직후] 테스트 정확도: {acc_inproc:.4f} ({int((pred == y_test).sum())}/{len(y_test)})")

    # 4) Keras v3 네이티브 포맷으로 저장 — 아키텍처+가중치+컴파일 설정 전부 포함
    # (scikit-learn의 pickle, PyTorch의 TorchScript와 대응되는 Keras만의 저장 방식)
    model.save("iris_model.keras")
    print("[Keras] 저장 완료: iris_model.keras")

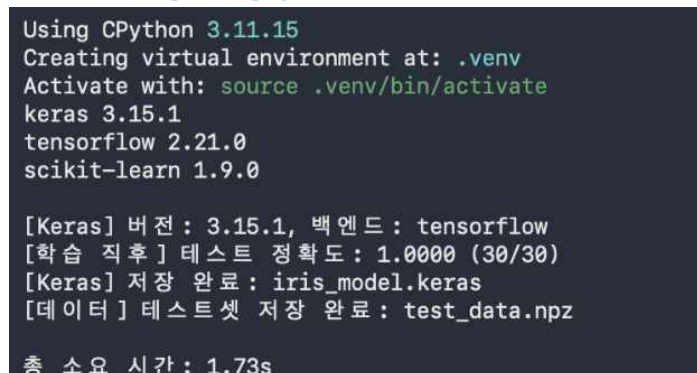
    # 5) 다음 단계(predict_from_saved.py, 별도 프로세스)가 재사용할 테스트 데이터·정확도 저장
    np.savez("test_data.npz", X_test=X_test_scaled, y_test=y_test, acc_inproc=acc_inproc)
    print("[데이터] 테스트셋 저장 완료: test_data.npz")

if __name__ == "__main__":
    t0 = time.time()
    main()
    print(f"\n총 소요 시간: {time.time() - t0:.2f}s")

```

command python train_and_save.py

[follow_01_setup_train.png](#)



```

Using CPython 3.11.15
Creating virtual environment at: .venv
Activate with: source .venv/bin/activate
keras 3.15.1
tensorflow 2.21.0
scikit-learn 1.9.0

[Keras] 버전 : 3.15.1, 백엔드 : tensorflow
[학습 직후] 테스트 정확도 : 1.0000 (30/30)
[Keras] 저장 완료 : iris_model.keras
[데이터] 테스트셋 저장 완료 : test_data.npz

총 소요 시간 : 1.73s

```

keras 복원 + predict + 비교

[base-keras/tpredict_from_saved.py](#)

저장된 iris_model.keras를 모델 정의 코드 없이 로드해 predict한다.

이 스크립트에는 build_model()도 keras.Sequential(...)도 없다.

별도 python 프로세스로 실행되므로 1단계에서 메모리에 있던 model 객체는 이미 사라진 상태다.

```
keras.models.load_model("iris_model.keras")
```

한 줄이 .keras 파일 안에 저장된 아키텍처+가중치+컴파일 설정을 전부 읽어 완전한 모델을 복원한다.

model.summary()가 이 프로세스가 정의하지 않은 구조(4→16→8→3)를 그대로 출력하는 것이 그 증거다.

```
import os
```

```
# 1단계와 동일하게 TensorFlow 백엔드 고정 (keras import 전에 설정)
```

```
os.environ.setdefault("KERAS_BACKEND", "tensorflow")
```

```
import time
```

```
import keras
```

```
import numpy as np
```

```
def main():
```

```
    model_path = "iris_model.keras"
```

```
    # 1) 테스트 데이터 로드 (1단계에서 저장한 것)
```

```
    data = np.load("test_data.npz")
```

```
    X_test = data["X_test"]
```

```
    y_test = data["y_test"]
```

```
    acc_inproc = float(data["acc_inproc"])
```

```
    # 2) 모델 복원 — 이 프로세스 어디에도 모델 구조를 정의하는 코드가 없다
```

```
    model = keras.models.load_model(model_path)
```

```
    print(f"[Keras] 로드 완료: {model_path} (백엔드={keras.backend.backend()})")
```

```
    model.summary() # 구조 정의 코드 없이도 4->16->8->3 구조가 그대로 출력됨
```

```
    # 3) predict 실행
```

```
    t0 = time.time()
```

```
    pred = model.predict(X_test, verbose=0).argmax(axis=1)
```

```
    latency = time.time() - t0
```

```
    acc_loaded = float((pred == y_test).mean())
```

```
    print(f"Wn[복원 모델] 테스트 정확도: {acc_loaded:.4f} ({int((pred == y_test).sum())}/{len(y_test)})")
```

```
    print(f"[복원 모델] predict 소요 시간 ({len(X_test)}개 샘플): {latency * 1000:.3f} ms")
```

```
    # 4) 1단계(학습 직후, 같은 프로세스)와 결과가 동일한지 검증
```

```
    print(f"Wn[비교] 학습 직후 테스트 정확도 : {acc_inproc:.4f}")
```

```
    print(f"[비교] .keras 복원 후 테스트 정확도 : {acc_loaded:.4f}")
```

```
    if abs(acc_inproc - acc_loaded) < 1e-9:
```

```
        print("[비교] 결과 일치 ☑ — 재학습도 모델 정의 코드도 없이 .keras 파일만으로 동일한 모델을 복원했다.")
```

```
    else:
```

```
        print("[비교] 결과 불일치 ⚠")
```

```
if __name__ == "__main__":
```

```
    main()
```

predict 소요 시간이 sklearn/PyTorch 예시보다 크게(예시치 42ms) 나올 수 있는데,

TensorFlow는 첫 predict 호출 때 내부 그래프 초기화 비용이 들어가기 때문이다.

```
command python predict_from_saved.py
```

[follow_02_predict_compare.png](#)

```
[Keras] 로드 완료 : iris_model.keras (백엔드=tensorflow)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 16)	80
dense_1 (Dense)	(None, 8)	136
dense_2 (Dense)	(None, 3)	27

```

Total params: 731 (2.86 KB)
Trainable params: 243 (972.00 B)
Non-trainable params: 0 (0.00 B)
Optimizer params: 488 (1.91 KB)

[복원 모델] 테스트 정확도 : 1.0000 (30/30)
[복원 모델] predict 소요 시간 (30개 샘플): 42.535 ms

[비교] 학습 직후 테스트 정확도      : 1.0000
[비교] .keras 복원 후 테스트 정확도  : 1.0000
[비교] 결과 일치  ✓ - 재학습도 모델 정의 코드도 없이 .keras 파일만으로 동일한 모델을 복원했다.

```

fastapi-keras3

.keras 모델을 FastAPI로 서빙

[fastapi-keras/followup.md](#) 순서대로 진행

Keras(TensorFlow 백엔드)로 학습한 모델을 .keras 파일로 저장한 뒤,
FastAPI 서버의 lifespan에서 그 파일을 프로세스 생애주기 동안 딱 한 번만 로드하고,
이후 들어오는 모든 /predict 요청이 그 하나의 모델 객체를 재사용하는지 확인한다.

"요청마다 모델을 다시 로드하지 않고도, 서로 다른 세 가지 입력을 전부 정확히 예측할 수 있는가?"

```

cd fastapi-keras
uv venv --python 3.11 --clear .venv
source .venv/bin/activate
uv pip install --quiet keras tensorflow scikit-learn numpy fastapi "uvicorn[standard]"

```

[fastapi-keras/train.py](#)

Keras(TensorFlow 백엔드)로 붓꽃(Iris) 분류 모델을 학습하고 저장한다.
base-keras와 같은 모델(keras.Sequential, 4->16->8->3)을 쓰되, 이번엔 FastAPI 서빙까지 포함한다.
학습 때 쓴 StandardScaler의 mean_/scale_도 scaler.npz로 함께 저장한다.
app.py(서빙 쪽)는 sklearn을 아예 import하지 않고 이 두 배열만으로 표준화를 재현한다.

FastAPI 서버 기동 + /health + /predict 3회 + 서버 종료

Keras(TensorFlow 백엔드)로 저장한 iris_model.keras를 FastAPI로 서빙한다.
KERAS_BACKEND는 keras를 import하기 전에 결정돼야 하므로,
이 파일 맨 위(다른 import보다 먼저)에서 환경변수를 강제로 tensorflow로 고정한다.
lifespan에서 model-scaler를 프로세스 시작 시 1회만 로드해 요청마다의 로딩 비용을 없앤다.
scikit-learn·PyTorch 서빙 예시와 동일한 설계.

```

import os
os.environ.setdefault("KERAS_BACKEND", "tensorflow")
from contextlib import asynccontextmanager
from time import perf_counter
import keras
import numpy as np
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

```

```

IRIS_CLASS_NAMES = ["setosa", "versicolor", "virginica"]
ml_state: dict = {}

```

```

@asynccontextmanager
async def lifespan(app: FastAPI):
    # startup: model-scaler를 프로세스 생애주기 동안 한 번만 로드

```

```

model = keras.models.load_model("iris_model.keras")
scaler = np.load("scaler.npz")
ml_state["model"] = model
ml_state["mean"] = scaler["mean"]
ml_state["scale"] = scaler["scale"]
print(f"[startup] Keras 모델 로드 완료 (keras {keras.__version__}, 백엔드={keras.backend.backend()})")

yield

# shutdown: 정리할 리소스 없음
ml_state.clear()

app = FastAPI(title="Iris Keras(TensorFlow backend) API", lifespan=lifespan)

class IrisFeatures(BaseModel):
    sepal_length_cm: float = Field(..., ge=0, description="꽃받침 길이(cm)")
    sepal_width_cm: float = Field(..., ge=0, description="꽃받침 너비(cm)")
    petal_length_cm: float = Field(..., ge=0, description="꽃잎 길이(cm)")
    petal_width_cm: float = Field(..., ge=0, description="꽃잎 너비(cm)")

class PredictResponse(BaseModel):
    predicted_class: str
    probabilities: dict
    latency_ms: float

def softmax(logits: np.ndarray) -> np.ndarray:
    # 모델 출력이 로짓(softmax 이전 값)이므로 서빙 단에서 직접 확률로 변환
    e = np.exp(logits - logits.max(axis=-1, keepdims=True))    # 오버플로 방지를 위해 최댓값을 뺀 뒤 exp
    return e / e.sum(axis=-1, keepdims=True)

@app.get("/health")
def health():
    if "model" not in ml_state:
        raise HTTPException(status_code=503, detail="model not loaded")
    return {
        "status": "ok",
        "keras_version": keras.__version__,
        "keras_backend": keras.backend.backend(),
    }

@app.post("/predict", response_model=PredictResponse)
def predict(features: IrisFeatures):
    raw = np.array(
        [[
            features.sepal_length_cm,
            features.sepal_width_cm,
            features.petal_length_cm,
            features.petal_width_cm,
        ]],
        dtype=np.float32,
    )

    # 학습 때와 동일한 표준화: (x - mean) / scale — sklearn 없이 numpy만으로 재현
    scaled = (raw - ml_state["mean"]) / ml_state["scale"]
    t0 = perf_counter()
    logits = ml_state["model"].predict(scaled, verbose=0)
    latency_ms = (perf_counter() - t0) * 1000
    probs = softmax(logits)[0]
    pred_idx = int(probs.argmax())

```

```

return PredictResponse(
    predicted_class=IRIS_CLASS_NAMES[pred_idx],
    probabilities={name: round(float(p), 4) for name, p in zip(IRIS_CLASS_NAMES, probs)},
    latency_ms=round(latency_ms, 4),
)

```

follow_01_setup_train.png

```

Using CPython 3.11.15
Creating virtual environment at: .venv
Activate with: source .venv/bin/activate
keras 3.15.1
tensorflow 2.21.0
fastapi 0.141.1
uvicorn 0.52.4

[Keras] 버전 : 3.15.1, 백엔드 : tensorflow
[Keras/tensorflow] 테스트 정확도 : 1.0000
[Keras] 저장 완료 : iris_model.keras
[Scaler] 저장 완료 : scaler.npz (mean, scale)

총 소요 시간 : 1.70s

```

```
command uvicorn app:app --host 127.0.0.1 --port 8341 > server.log 2>&1 &
```

```

# 서버가 뜰 때까지 대기 (최대 20초)
for i in $(seq 1 40); do
    if curl -s -o /dev/null "http://127.0.0.1:8341/health"; then break; fi
    sleep 0.5
done

curl -s "http://127.0.0.1:8341/health" | command python -m json.tool

curl -s -X POST "http://127.0.0.1:8341/predict" \
    -H "Content-Type: application/json" \
    -d '{"sepal_length_cm": 5.1, "sepal_width_cm": 3.5, "petal_length_cm": 1.4, "petal_width_cm": 0.2}' \
    | command python -m json.tool
# versicolor(7.0,3.2,4.7,1.4), virginica(6.3,3.3,6.0,2.5)도 같은 방식으로 확인

```

```
cat server.log
```

```

# 확인이 끝나면 서버 종료
lsof -ti:8341 | xargs kill -9

```

follow_02_serve_predict.png

```

--- GET /health ---
{
  "status": "ok",
  "keras_version": "3.15.1",
  "keras_backend": "tensorflow"
}

--- POST /predict x3 ---
[setosa] predicted_class=setosa top_prob=1.0000 latency_ms=57.8766
[versicolor] predicted_class=versicolor top_prob=1.0000 latency_ms=13.4419
[virginica] predicted_class=virginica top_prob=1.0000 latency_ms=13.9032

--- server.log (startup 로그 확인) ---
INFO: Started server process [42957]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8341 (Press CTRL+C to quit)
[startup] Keras 모델 로드 완료 (keras 3.15.1, 백엔드=tensorflow)
INFO: 127.0.0.1:62964 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:62965 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:62966 - "POST /predict HTTP/1.1" 200 OK
INFO: 127.0.0.1:62967 - "POST /predict HTTP/1.1" 200 OK
INFO: 127.0.0.1:62968 - "POST /predict HTTP/1.1" 200 OK

=== 서버 종료 ===
/private/tmp/claude-502/-Users/manure--work-re-scala/154b049b-1078-42ba-a77b-7af3be21cbc4/scratchpad/fk_node2_serve_predict.sh: line 27: 42957 Killed: 9
command uvicorn app:app --host 127.0.0.1 --port 8341 > server.log 2>&1
서버 종료 완료

```



수고하셨습니다.

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

